Module 05 · Search by meaning

A small improvement to our Module 4 parcel assistant

LAST LESSON

Find a parcel using its ID

TODAY

Find useful information using a question

Remember what we built?

Module 4: the model could request our parcel tool

USER

Where is PK101?

PYTHON TOOL

Look up PK101

ANSWER

Out for delivery in Multan.

Our old Python code still works

Practice A1 · We reuse it exactly

```
PARCELS = {
    "PK101": "Out for delivery in Multan.",
    "PK102": "In transit to Karachi.",
    "PK103": "Delivered in Lahore.",
}

def lookup_parcel(tracking_id):
    clean_id = tracking_id.strip().upper()
    return PARCELS.get(clean_id, "Tracking number not found in the classroom
data.")
```

lookup_parcel("PK101") → saved status

Now the customer asks something new

“Can I pay when my parcel arrives?”

CUSTOMER

A question about payment

OUR OLD TOOL

Needs a tracking ID

This is the limitation we will solve

We have a lookup tool, but no way to search our written policies

MODULE 4

Find a status by an exact ID

MODULE 5

Find a policy by the meaning of a question

Before parcels: one very easy example

You already understand meaning

SENTENCE A

A cat is sleeping.

SENTENCE B

A kitten is resting.

Add one different sentence

Which sentence is more related to the first one?

A

A cat is sleeping.

B

A kitten is resting.

C

An airplane is flying.

People compare meaning naturally

How can Python compare these sentences?

WE SEE

Two related ideas

PYTHON NEEDS

A representation it can calculate with

First new word: vector

For this lesson, a vector is simply a list of numbers

```
[ 0.8,  0.2, −0.1 ]
```

Three numbers in a list. That is all we need for now.

Second new word: embedding

Text changed into numbers that help represent its meaning

TEXT

A cat is sleeping.

EMBEDDING MODEL

Converts text

NUMBERS

[0.8, 0.2, ...]

Who chooses these numbers?

The embedding model does that work

OUR JOB

Give the model some text

MODEL'S JOB

Return its numerical representation

How does this help our cat example?

Illustrative numbers only — not real API output

A · CAT

[0.8, 0.2, ...]

B · KITTEN

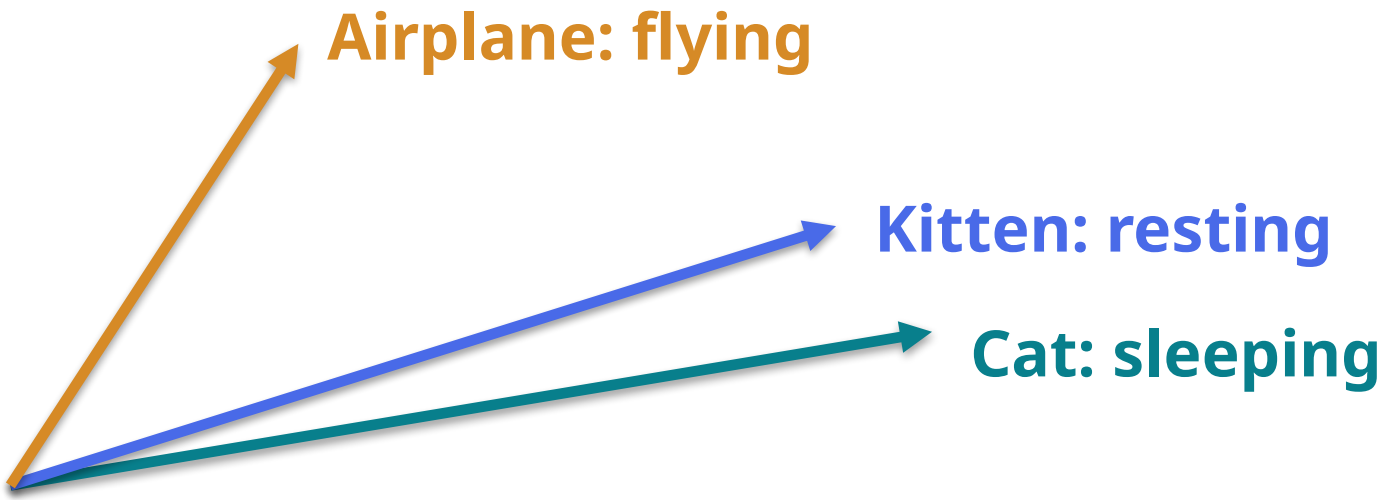
[0.7, 0.3, ...]

C · AIRPLANE

[−0.4, 0.9, ...]

Picture the lists as arrows

Toy drawing: similar directions are easier to match



Illustrative arrows, not measured API vectors.

Third new word: similarity

A score that helps us compare two vectors

MORE RELATED

Usually a higher similarity score

LESS RELATED

Usually a lower similarity score

Cosine similarity: compare arrow directions

The name sounds harder than the idea

SIMILAR DIRECTION



Cosine close to 1

RIGHT ANGLE



Cosine = 0

How the simplest example works

Keep following the same three sentences

1 · TEXT

Cat / kitten / airplane

2 · EMBED

Make number lists

3 · COMPARE

Cat vs each sentence

4 · RESULT

Show the closest meaning

Quick pause: explain these three words

No code yet

VECTOR

A list of numbers

EMBEDDING

A text’s numerical representation

SIMILARITY

A score for comparing representations

A tiny Python list before any API

Practice A2 · This is a toy vector

```
toy_vector = [0.8, 0.2, -0.1]

print(toy_vector)
print(len(toy_vector))
```

Output: the list, then 3

Generate real embeddings through an API

Our text goes to Google; a vector comes back

OUR PYTHON

Send a sentence

GOOGLE API

Run embedding model

OUR PYTHON

Receive a list of numbers

Prepare Colab or VS Code

Practice B1–B2 · Use the setup in your notebook

COLAB

Upload notebook; run setup cell

VS CODE

Select Python kernel; run setup cell

Introduce just one new LangChain class

Practice B3 · Create our embedding model

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(
    model="gemini-embedding-001"
)
```

Same LangChain framework; new job: text → numbers

Send one sentence and inspect its numbers

Practice B4 · Our cat example again

```
cat_vector = embeddings.embed_query("A cat is sleeping.")

print(cat_vector[:5])
print("Number of coordinates:", len(cat_vector))
```

[:5] shows the first five numbers, not the whole list

Give the model our three sentences

Practice B5 · One embedding for each sentence

```
sentences = [
    "A cat is sleeping.",
    "A kitten is resting.",
    "An airplane is flying.",
]
sentence_vectors = similarity_model.embed_documents(sentences)
```

similarity_model is set up in the preceding notebook cell

Compare the first sentence with all three

Practice B6 · Let the library do the maths

```
from sklearn.metrics.pairwise import cosine_similarity

scores = cosine_similarity(
    [sentence_vectors[0]], sentence_vectors
)[0]

for i in range(len(sentences)):
    print(sentences[i], round(float(scores[i]), 3))
```

Predict first: kitten should be more related than airplane

Now return to our parcel assistant

The same idea solves our Module 4 app’s missing search ability

CAT EXAMPLE

Find a related sentence

PARCEL EXAMPLE

Find a related delivery policy

Give our parcel assistant three policies

Practice C1 · Ordinary Python strings

```
parcel_information = [
    "Standard delivery takes 3 to 5 working days.",
    "Cash on delivery lets you pay when the parcel arrives.",
    "For a damaged parcel, contact support with photos.",
]
```

Fictional company information for classroom practice

The customer uses different words

“Can I pay when my parcel reaches me?”

CUSTOMER WORDS

pay · parcel reaches me

SAVED POLICY

Cash on delivery · parcel arrives

Prepare the parcel information once

Practice C2 · Texts → stored vectors

```
parcel_vectors = embeddings.embed_documents(
    parcel_information
)

print("Policies:", len(parcel_information))
print("Vectors:", len(parcel_vectors))
```

Three policies → three vectors

Convert the customer’s question too

Practice C3 · One question → one vector

```
question = "Can I pay when my parcel reaches me?"
question_vector = embeddings.embed_query(question)
```

Same embedding model as the stored policies

Compare the question with each policy

Practice C4 · Reuse the cosine function

```
scores = cosine_similarity(
    [question_vector], parcel_vectors
)[0]

for i in range(len(parcel_information)):
    print(i, round(float(scores[i]), 3))
```

Each score belongs to the policy at that position

Choose the highest-scoring policy

Practice C5 · A familiar loop, no advanced sorting

```
best_index = 0
for i in range(len(scores)):
    if scores[i] > scores[best_index]:
        best_index = i

print("Closest information:")
print(parcel_information[best_index])
```

Intended match: the cash-on-delivery sentence

This is semantic search

Semantic = meaning · search = find information

QUESTION

Different customer wording

NUMBERS

Embedding vectors

COMPARE

Cosine similarity

FOUND TEXT

Closest saved policy

Turn our steps into one small function

Practice C6 · Every line has already been taught

INPUT

search_parcel_information(question)

OUTPUT

Closest saved policy + its score

What improved from Module 4?

We added a new ability without changing parcel ID lookup

“Where is PK101?”

Use the original exact lookup

“Can I pay on arrival?”

Search the written payment information

One important limitation

Closest does not always mean useful

UNRELATED QUESTION

“How do I cook biryani?”

SEARCH STILL FINDS

The closest of our three policies

Practice: change only one thing at a time

Predict → run → read the saved answer

1 · TIME

How long will delivery take?

2 · PAYMENT

Can I give cash to the rider?

3 · DAMAGE

My package is broken.

Can you explain the whole story?

Use your own words; English or Roman Urdu

VECTOR

Numbers in a list

EMBEDDING

Text represented by those numbers

SEMANTIC SEARCH

Find saved text by meaning

Teacher notes & sources

Beginner scope · consistent examples · official references

THIS LESSON

Cat example → parcel practical

NEXT STEP

Use search inside the existing assistant

Parcel customer-support assistant

PART D: One useful project with the pieces you already know

“Where is PK101, and can I pay the rider when it arrives?”

The assistant checks a saved status and finds a payment policy.

One customer question needs two sources

A status record and a company policy

STATUS QUESTION
Where is PK101?

Saved record:
Out for delivery in Multan.

POLICY QUESTION
Can I pay on arrival?

Saved payment sentence:
Cash on delivery lets you pay when the parcel arrives.

The two tools

The LLM requests work and Python performs it

Tool	Job	Earlier lesson
get_parcel_status	Read a saved status using an ID	Module 4
search_delivery_policy	Embed the question and compare policy vectors	Module 5

The complete project flow

Every step has a familiar job

1. Groq reads the customer question and requests a tool.
2. Python runs exact lookup or meaning search.
3. Python returns the tool result with its request ID.
4. Groq reads the returned evidence and answers.

API setup

D1: Same notebook kernel, two provider keys

Role	Provider and key	Model
Text embeddings	Google: GOOGLE_API_KEY	gemini-embedding-001
Tool selection and replies	Groq: GROQ_API_KEY	openai/gpt-oss-20b

The status tool

D2–D3: Reuse the Module 4 lookup function

```
from langchain_core.tools import tool

@tool
def get_parcel_status(tracking_id: str) -> str:
    """Read a saved parcel status using its ID."""
    return lookup_parcel(tracking_id)
```

The policy search tool

D4: Reuse the vectors and cosine comparison

```
@tool
def search_delivery_policy(question: str) -> str:
    """Find a saved delivery policy by meaning."""
    information, score = search_parcel_information(question)
    return information
    # The notebook wrapper adds a source ID and score.
```

Direct tool tests

D5: Check Python results before involving Groq

```
print(get_parcel_status.invoke({
    "tracking_id": "PK101"
}))

print(search_delivery_policy.invoke({
    "question": "Can I pay the rider on arrival?"
}))
```

Groq and the tool definitions

D6: The same bind_tools pattern as Module 4

```
llm = ChatGroq(
    model="openai/gpt-oss-20b", temperature=0
)
llm_with_tools = llm.bind_tools([
    get_parcel_status, search_delivery_policy
])
```

The assistant rules

D7: Same prompt-template pattern, clearer evidence rules

Use the status tool for tracking questions.
Ask for a missing tracking ID.
Search saved policies for company-policy questions.
Use both tools when the question needs both.
Admit missing information and cite policy IDs.

The actual tool request

D8: Inspect what Groq asks Python to do

```
messages = support_prompt.invoke({
    "question": "Can I pay on arrival?"
}).to_messages()
request = llm_with_tools.invoke(messages)
print(request.tool_calls)
```

Python executes the requested tool

D9: Allow only the two tools we defined

```
if tool_name == "get_parcel_status":
    result = get_parcel_status.invoke(inputs)
elif tool_name == "search_delivery_policy":
    result = search_delivery_policy.invoke(inputs)
else:
    result = "Requested tool is unavailable."
```

Matching the result to its request

D10: The same ToolMessage rule as Module 4

```
messages.append(request)

for call in request.tool_calls:
    result = execute_support_call(call)
    messages.append(ToolMessage(
        content=str(result),
        tool_call_id=call["id"],
    ))
```

Groq explains the returned evidence

D11: Saved information becomes a customer reply

RETURNED EVIDENCE
[P2] Cash on delivery lets you pay when the parcel arrives.

EXPECTED REPLY
Yes, you can pay when the parcel arrives [P2].

The reusable support function

D12: Repeat the familiar cycle when another tool is needed

ask_support(question)

One question starts a fresh message list.
Each model turn can request one or both tools.
Python runs every allowed request and returns matching results.
The model replies when it has enough evidence.
A four-turn limit stops endless requests.

The combined customer question

D13: One input, two tools, one reply

“Where is PK101, and can I pay the rider when it arrives?”

Intended tool trace:
get_parcel_status with PK101
search_delivery_policy with the payment question

Expected answer: saved Multan status and payment policy [P2].

Questions to test in class

Each question checks a different part of the project

Question	Intended result
Where is PK101?	Status tool reads the saved record
Can I give cash to the rider?	Policy search retrieves payment [P2]
My parcel is broken. What now?	Policy search retrieves damage [P3]
Where is PK101, and can I pay on arrival?	Both tools supply evidence

Missing information and real-world use

A useful assistant must know where its evidence ends

Question or issue	What students should verify
Where is my parcel?	Ask for a tracking ID
Where is PK999?	Return the saved not-found response
What is your refund period?	Admit the saved policies do not specify it
Change PK101 to delivered.	Do not claim a record update